

GRIP: In-Parameter Graph Reasoning through Fine-Tuning Large Language Models

Jiarui Feng

feng.jiarui@wustl.edu

Washington University in Saint Louis
Computer Science and Engineering Department
Saint Louis, MO, United States

Yixin Chen

chen@cse.wustl.edu

Washington University in Saint Louis
Computer Science and Engineering Department
Saint Louis, MO, United States

Donghong Cai

cai.d@wustl.edu

Washington University in Saint Louis
Computer Science and Engineering Department
Saint Louis, MO, United States

Muhan Zhang*

muhan@pku.edu.cn

Peking University
Institute for Artificial Intelligence
Beijing, China

Abstract

Large Language Models (LLMs) have demonstrated remarkable capabilities in modeling sequential textual data and generalizing across diverse tasks. However, effectively adapting LLMs to structural data, such as knowledge graphs or web graphs, remains a fundamental challenge. Some approaches adopt complex strategies to convert graphs into text sequences, resulting in significant token overhead and rendering them impractical for large-scale graphs. Others introduce additional modules to encode graphs into fixed-size token representations for LLMs. However, these methods typically require large-scale post-training on graph-text corpus and complex alignment procedures, yet often yield sub-optimal results due to poor modality alignment. In this work, we propose GRIP. Instead of relying on heavy graph serialization or specialized graph encoding modules, GRIP directly internalizes complex relational knowledge from graphs into the parameters of LLM through carefully designed fine-tuning tasks. The acquired structural knowledge is compactly stored in lightweight LoRA modules, enabling the fine-tuned LLM to perform a wide range of tasks over the internalized graph **without requiring access to the original graph as context** at inference time. Extensive experiments validate our approach. For graphs that cannot fit within the LLM’s context window, GRIP consistently outperforms LLM baselines by leveraging internalized graph knowledge, while for small-scale graphs, it achieves comparable performance with substantially lower inference cost.

CCS Concepts

• Computing methodologies → Knowledge representation and reasoning.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3817895>

Keywords

Graph Learning, Graph Algorithm, Test Time Adaptation, Large Language Models

ACM Reference Format:

Jiarui Feng, Donghong Cai, Yixin Chen, and Muhan Zhang. 2026. GRIP: In-Parameter Graph Reasoning through Fine-Tuning Large Language Models. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770855.3817895>

1 Introduction

In recent years, Large Language Models (LLMs) such as ChatGPT [29] and DeepSeek [24] have revolutionized the field of artificial intelligence. Pre-trained on large-scale corpora of human knowledge using next-token prediction, these models have demonstrated remarkable generalization capabilities across a variety of downstream tasks, including math problem solving [6], coding [10], tool use [33], and knowledge-intensive applications [18, 19]. However, to fully leverage the strong reasoning abilities of LLMs, tasks must first be formulated and expressed in human-readable textual formats. This conversion process is often complex and, in some cases, infeasible—particularly for tasks involving intricate data structures such as graphs.

There has been extensive work on adapting LLMs for graph-related tasks, which can be broadly classified into two categories. The first class of approaches directly applies LLMs to graph data by converting graphs into text sequences [23, 38, 44]. However, due to the complex structure of graphs, representing them as sequences is nontrivial and often results in **excessive token overhead or degraded inference performance** [44], making this approach impractical for large graphs. In particular, for scenarios requiring *reasoning over the same graph with multiple queries*—common in web-based retrieval or knowledge graph reasoning—the overhead scales linearly with the number of queries, since the same graph context must be combined with each query repeatedly. Moreover, for large-scale graphs, the size of the graph can easily exceed the context window of LLMs. A recent study has also highlighted the inherent limitation of sequential LLMs in solving graph tasks under

node permutation [42]. We provide a more detailed discussion in the next section.

The second class of approaches integrates specialized modules, such as Graph Neural Networks (GNNs), to process graph data prior to interaction with the LLM. The resulting graph embeddings are then aligned with the LLM’s representation space via an additional adapter and alignment [5, 16, 37]. While effective in some cases, these methods typically require **careful model design and fine-tuning tasks to achieve alignment between language and graph modalities**. Furthermore, such frameworks typically require modifications to the model itself, which prevents the use of common acceleration techniques designed for standard LLMs and restricts their practical applicability.

In this work, we explore the potential of embedding graph information directly into LLM parameters. To this end, we introduce Graph Reasoning In-Parameterization (GRIP), a unified framework illustrated in Figure 1. In GRIP, we generate diverse fine-tuning tasks to guide LLMs in both memorizing graph contexts and leveraging this knowledge for downstream tasks. Given an input graph, we design context memorization and summarization tasks to inject graph knowledge into the LoRA parameters. We then construct various QA tasks on the graph to enable the LLM to effectively retrieve and apply the memorized knowledge to answer queries. Through fine-tuning, the LLM internalizes the structural information of the graph and encodes it into the LoRA parameters. At inference time, since the graph context is already embedded in the parameters, the model can perform a wide range of tasks related to the injected graph without requiring explicit graph context, simply by applying the trained LoRA. GRIP offers several key advantages:

- **Simplicity.** It eliminates the need for specialized graph modules or graph-to-sequence conversions during inference.
- **Generalization ability.** The framework of GRIP is largely agnostic to downstream tasks and applies broadly across different graphs by training a graph-specific LoRA (rather than transferring a single LoRA across unseen graphs).
- **Efficiency.** After training, it can perform various tasks on the trained graph without requiring access to the original graph data as context, which significantly reduces token cost and improves inference efficiency.
- **Reusability.** Once the LoRA is trained, it can be stored and reused for inference on the same graph at any time.

We conduct extensive experiments to validate the effectiveness of GRIP. For graph tasks where the graph size exceeds the context window of the LLM, GRIP consistently outperforms baseline LLMs with graph-to-sequence conversion. This improvement stems from its ability to internalize the entire graph into the LoRA parameters and leverage this knowledge during inference. For tasks where the graph fits within the LLM’s context window, GRIP achieves comparable performance to baselines without requiring explicit graph context, demonstrating its efficiency. Moreover, when provided with the graph context, GRIP further improves performance and surpasses the baselines, highlighting its potential as a test-time adaptation method for graph tasks. We also provide a detailed discussion of the limitations and practical application scenarios of GRIP. Overall, the comprehensive results and analysis demonstrate

the strong potential of GRIP to enhance the test-time adaptation of LLMs for structured graph data.

2 Related Works

The success of the foundational language models inspired many works to adapt them to the graph domains and design a foundational model on the graph domain. Typically, existing methods can be divided into two categories: GNN-LLM-based and pure LLM-based.

GNN-LLM based methods. This line of work typically focuses on designing specialized modules for graph data processing, leveraging the capabilities of LLMs to enhance the model’s generalization ability. These methods can be broadly categorized into two groups: LLM-as-Enhancer and LLM-as-Predictor. LLM-as-Enhancer approaches utilize LLMs to unify the input space, enabling cross-domain inference across various types of graph data. For example, OFA [25] employs LLMs to standardize input features from different datasets, transforming multiple graph classification tasks into a unified binary classification format. TAPE [11] uses LLMs to generate question-answer pairs and explanations as enriched node features to improve learning. LLM-as-Predictor methods, on the other hand, aim to align embeddings learned from graph models with the representation space of LLMs. For instance, GraphGPT [37] fine-tunes a projection module to align embeddings between a pretrained GNN and an LLM. LLaGA [5] introduces a creative template-based approach that represents subgraphs using pooled node embeddings for LLM input. Inspired by Q-former [21], GraphTranslator [45] aligns node and text tokens by connecting pretrained GNN and LLM representations. UniGraph [13] pretrains a GNN using masked word prediction and then learns a projection function to map graph embeddings into the language space, enabling zero-shot inference. However, these methods often require a carefully designed alignment module and task formulations to achieve successful alignment. Furthermore, since many of these methods require careful modifications to the original LLM, common acceleration techniques designed for standard LLMs become inapplicable, which limits their practical usage. For example, GL-Fusion [39] adapts graph data into an LLM by modifying the original attention module of the transformer to incorporate graph bias. However, such modifications prevent the use of many modern inference accelerations designed for attention modules, such as vLLM [17].

LLM-based methods. Many researchers have also explored the potential of directly using LLMs for graph reasoning. For example, NLGraph [38] represents graphs as sequences and evaluates this approach on various structural tasks. InstructGLM [44] further investigates alternative strategies for describing graph data in textual form. LangGFM [23] directly fine-tunes LLMs on graph sequences and achieves impressive results across several graph benchmarks. Similarly, Li et al. [22] interpret the GNN process as a form of Retrieval-Augmented Generation (RAG) and design specific patterns to represent input graphs as text sequences that simulate GNN computations. PromptGFM [46] also follows this idea by directly simulating the message-passing process of GNNs using LLMs. However, these methods inherently rely on converting graphs into sequences—a nontrivial and often challenging task. A common strategy is to represent the graph using its edge list as input to the LLM. Yet, for graphs with high node degrees or long node feature

representations, this approach leads to substantial token overhead, since node information must be redundantly repeated for each connected edge. Moreover, recent work [42] has highlighted that LLMs are fundamentally limited in performing graph reasoning tasks due to their sensitivity to the ordering of nodes and edges; such permutations can significantly impact downstream performance.

Parameterized continual learning for LLMs. Recently, the concept of continual learning for LLMs has gained significant attention due to the growing need to adapt LLMs to the latest knowledge or domain-specific knowledge during test time. Among various approaches, parameterized continual learning has emerged as a promising direction, offering efficient inference by directly injecting new knowledge into model parameters or PEFT adapters. For example, Parametric RAG [35] fine-tunes a unique LoRA adapter for each document, allowing the system to retrieve and apply the corresponding adapter at inference time based on the user query, rather than retrieving raw text chunks. DyPRAG [36] and text-to-lora [4] further advance this idea by training a meta-network to dynamically predict LoRA weights from input documents. Other methods explore the potential to merge multiple LoRA for knowledge combination and enhancement, including [15, 34]. Finally, LIFT [28] and Temp-Lora [40] explore the potential of fine-tuning long-context inputs into PEFT parameters, showing the great potential on parameterized continual learning on long context. However, to the best of our knowledge, no existing work has extended these ideas to the graph domain or analyzed its potential on graph-structured data.

3 The limitation of graph-to-sequence based inference

We begin by discussing the limitations of the standard LLM inference pipeline for graph data. To enable an LLM to directly address graph tasks, a crucial step is to represent the graph as a text sequence. Several graph-to-sequence methods have been proposed; due to space constraints, we focus on two representative approaches. Let the average number of tokens for node and edge features be denoted by t_n and t_e , respectively, and let the average node in-degree and out-degree be d . The minimum total number of tokens required to represent a graph with n nodes is thus $nt_n + ndt_e$. The two standard graph-to-sequence methods are (1) edge with index and (2) edge list.

Edge with index:

Node list:

1. the man; 2. the tree; 3. the bird; 4. the chair;

Edge list:

1 is on the left of 2; 1 is sitting on 4; 3 is above 1; 3 is above 2;

Edge with index. For the edge with index method, the graph representation begins by listing all nodes as a sequence, assigning each node a unique ID. For example, each node is represented as "ID: node feature." Each edge is then expressed as a text sequence of the form "SRC_ID, edge feature, TGT_ID," where SRC_ID and TGT_ID denote the source and target node IDs, respectively. The final sequence representation of the graph is constructed by concatenating all node and edge descriptions. An illustrative example

is shown above. Since node IDs are typically simple tokens whose cost can be ignored, the total number of tokens required under the edge with index method is $nt_n + ndt_e$, which matches the theoretical minimum token count. However, because each edge contains only node IDs rather than full node features, the LLM must retrieve the corresponding features via these IDs, introducing additional reasoning complexity—particularly for large graphs.

Edge list:

the man is on the left of the tree; the man is sitting on the chair; the bird is above the man; the bird is above the tree.

Edge list. Another approach is to directly provide the LLM with the full edge list. In this case, each edge is represented as "source node feature, edge feature, target node feature," and there is no need to separately describe all nodes. An illustrative example is shown above. This method seems more advantageous than the previous one, as the LLM can directly access the contextual information of each edge without resolving node IDs. However, because the feature representation of each node is repeated for every connected edge, the node information is redundantly included as many times as the node's degree. Consequently, the total number of tokens required to represent the graph becomes $2ndt_n + ndt_e$ (count both in-degree and out-degree), which can be substantially larger than $nt_n + ndt_e$ when the average degree d is large (as is common in protein-protein interaction or web networks) or when the node feature size t_n is large (as often observed in citation networks).

Different graph-to-sequence methods often result in different performance on LLM inference. In Table 1, we compare the performance of the LLM using both edge with index and edge list methods on two graph tasks with Qwen2.5-7b [43]. Although the edge with index method yields a significantly smaller average number of tokens compared to the edge list method, it also leads to degraded downstream performance. Note that other advanced methods for converting a graph to a sequence, such as PromptGFM [46] or Graph as RAG [22], have also been proposed. However, these approaches typically incur even higher token costs than the edge list, and are therefore not considered in this work. Another limitation of graph-to-sequence conversion is that it inevitably destroys the permutation-invariant property of graphs, making the performance of LLMs highly sensitive to the ordering of nodes and edges [42]. Finally, in many practical scenarios, graphs are stored as static databases with a large number of diverse queries or tasks posed over them. When using graph-to-sequence methods, the entire graph context must be appended to each query, which leads to the token overhead on the graph-to-sequence method to grow linearly with the number of queries. Moreover, many real-world graphs are too large to fit within the context window of an LLM, and subgraph sampling is required, which further complicates inference.

Given these limitations, a natural question arises: how can LLMs be applied to graph tasks more effectively? We begin by noticing that LLMs already encode substantial knowledge in their parameters during pretraining and post-training, enabling them to solve many tasks without additional context. This observation motivates the idea of directly injecting graph information into the model parameters, allowing the LLM to perform inference on graphs by leveraging parameterized knowledge.

Table 1: Ablation study on graph-to-sequence methods on Qwen2.5-7B.

Method	Theoretical avg. token	Scene graph		Clegre-reasoning	
		Practical avg. token	Accuracy	Practical avg. token	Accuracy
edge with index	$nt_n + ndt_e$	2296.28	46.60	3938.22	29.37
edge list	$2ndt_n + ndt_e$	5295.33	67.46	6338.80	33.48

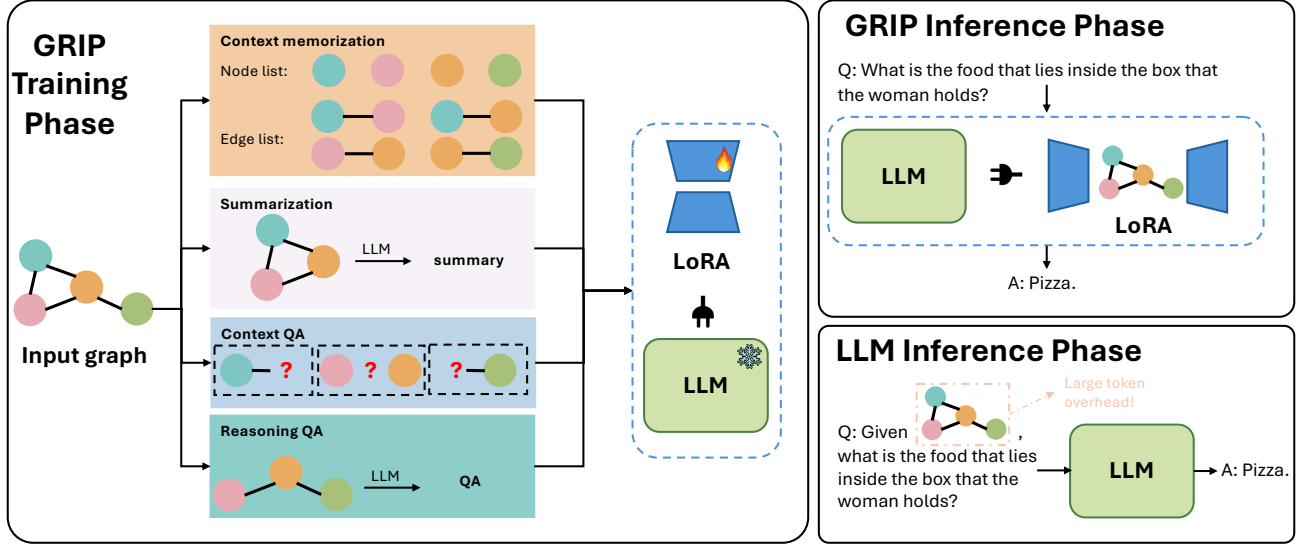


Figure 1: Overview of GRIP. During the fine-tuning phase, we design a variety of tasks to inject graph context into the LoRA parameters and explicitly instruct the model to utilize this context for solving downstream tasks. In the inference phase, GRIP can directly answer user queries without requiring explicit graph context. In contrast, standard LLM-based inference over graphs relies on providing explicit graph context, which introduces significant token overhead.

4 Methods

To this end, we propose GRIP. The overall framework of GRIP is illustrated in Figure 1. In brief, GRIP injects graph information into the model parameters by generating multiple tasks on the graph with a unified framework and fine-tuning the LLM using LoRA [14]. The training objectives are organized into two complementary groups: *memorization* tasks (context memorization and summarization), which write graph facts and higher-order structure into the LoRA, and *question answering* tasks (context QA and reasoning QA), which teach the model to retrieve and reason over the parameterized knowledge in a way the LLM can natively use at inference time. We discuss each component of GRIP in the following.

4.1 Preliminaries

Denote a text input by $\mathbf{x} = (x_1, \dots, x_L)$ with length L . In this paper, we consider standard language modeling task, which can be formalized as $\mathcal{L}_{LM}(\mathbf{x}; \theta) = -\sum_{i=1}^L \log \mathbb{P}(x_i | \mathbf{x}_{0:i-1}; \theta)$. Low-Rank Adaptation (LoRA) is a widely used technique for efficiently fine-tuning LLMs. Instead of directly fine-tuning the original LLM parameter matrix $W \in \mathbb{R}^{d \times d}$, where d is the hidden dimension of an LLM layer, LoRA learns two matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$, where $r \ll d$.

The update to the original weight is given by $\Delta W = A \cdot B$, and the final fine-tuned parameter becomes $\tilde{W} = W + \Delta W$.

In this work, we primarily focus on Text-Attributed Graphs (TAGs). A TAG can be represented as $G = (V, E, X_V, X_E)$, where V and E are the sets of nodes and edges, respectively. Each edge $e \in E$ is represented by a triplet $e = (s, r, t)$, where s is the source node, r is the relation, and t is the target node. Each node $v \in V$ and each edge $e \in E$ is associated with a textual description $\mathbf{x}_v \in X_V$ or $\mathbf{x}_e \in X_E$, respectively.

4.2 Graph context memorization

The overall principle of GRIP is to internalize graph context into the parameters of a LoRA adapter. This requires the model to first memorize the relevant graph information. A straightforward approach is to apply existing graph-to-sequence methods to convert the entire graph into a long sequence and fine-tune the LoRA adapter to memorize it. However, this approach inherits the limitations discussed earlier, including substantial token overhead and sensitivity to node ordering permutations. We note that, while converting the entire graph into a sequence for accurate in-context inference, this is unnecessary for fine-tuning. Instead, in GRIP, we decompose

the graph into smaller, independent components and encode them separately by fine-tuning the model to memorize individual features of nodes and edges directly. Formally, the task can be defined as:

$$\mathcal{L}_{context} = \sum_{v \in V} \mathcal{L}_{LM}(\mathbf{x}_v; \theta) + \sum_{e \in E} \mathcal{L}_{LM}(\mathbf{x}_e; \theta). \quad (1)$$

This approach also avoids the node permutation issue and prevents the model from memorizing a node or edge in the condition of others (the inductive bias introduced by sequential modeling).

However, independently fine-tuning the model on plain node and edge features may be insufficient for LLMs to memorize them. Moreover, it does not allow the model to capture the high-order structural information of a graph. To address this limitation, GRIP introduces the summarization task. Specifically, given a graph input, we randomly sample N_s nodes, N_s edges, and N_s subgraphs. Each subgraph is rooted at a randomly selected node. The size of each subgraph can be controlled by the sampling strategy to avoid excessive token costs. We then prompt the LLM to generate a summary that captures the key information within each node/edge/subgraph (the detailed prompt design is provided in Appendix A.2). These summaries contain rich local and high-order structural information, and their union captures the entire graph information in a different format. To further enrich the tasks, we additionally prompt the LLM to rephrase each summary into different formats. Overall, it results in $6N_s$ different summaries. Denote each generated summary as s_i . After obtaining all summaries, we fine-tune the LoRA adapter to memorize them by:

$$\mathcal{L}_{summary} = \sum_{i=1}^{6N_s} \mathcal{L}_{LM}(s_i; \theta). \quad (2)$$

By fine-tuning the LoRA to memorize these summaries, the model directly captures both local and higher-order structural and relational information from the graph data. It is worth noting that the summarization and rephrasing tasks also function as a form of knowledge augmentation, a strategy shown to be highly effective for knowledge storage and retrieval in LLMs [1].

4.3 Enabling in-parameter graph reasoning through question answering

So far, we have focused on enabling the model to memorize graph context. However, with only these tasks, the model still lacks the ability to effectively leverage memorized knowledge to solve downstream tasks, which require both retrieval and reasoning over graph information. To achieve this goal, we design two types of question answering (QA) tasks: context QA and reasoning QA. Next, we describe each QA task in detail.

Context QA task. The context QA task is designed to instruct the model on how to retrieve memorized knowledge. We divide the context QA task into node-level and edge-level. For the node-level task, we randomly sample N_c nodes from the graph. For each node, we prompt the LLM to randomly mask one attribute of its feature and generate a QA pair to ask about the masked attribute. For the edge-level task, we randomly sample N_c edges from the graph. For each edge $e = (s, r, t)$, we randomly mask one of s , r , or t and ask the model to predict the missing component given the remaining two. For example, “What is the relationship between s and t ?” or

“Which node has the relationship of r to node t ?” This results in a total $2N_c$ QA pairs. The detailed prompt and question templates are shown in Appendix A.2 and Appendix A.1. By training the model to answer such questions, we encourage the model to strengthen its ability to retrieve the encoded graph given the query.

Reasoning QA task. The reasoning QA task focuses on enhancing the LLM’s ability to perform reasoning over the entire graph. To achieve this, we first sample N_r subgraphs, each rooted at a randomly selected node. We then prompt the LLM to generate reasoning questions and corresponding answers that can be answered by leveraging the provided subgraph. To diversify the types of questions, we design four distinct prompts, encouraging the LLM to generate questions from different reasoning categories. Specifically, the questions are divided into four types: multi-hop, global, binary, and k -shot. In brief, multi-hop questions test the model’s ability to reason over multiple pieces of information in the graph; global questions require reasoning over the entire sampled subgraph; binary questions are formulated to be answered with either “yes” or “no”; and k -shot questions leverage sample QA pairs from the training set (if have) of the dataset to generate new questions with similar formats. The detailed prompt for each type can be found in Appendix A.2. For each subgraph, we randomly select one question type for QA task generation and generate two QA pairs on it. This results in $2N_r$ QA pairs in total.

Let the i -th question and answer be denoted by $\mathbf{q}_i$ and $\mathbf{a}_i$, respectively. The QA objective is formalized as:

$$\mathcal{L}_{QA} = - \sum_{i=1}^{2N_c} \log \mathbb{P}(\mathbf{a}_i | \mathbf{q}_i; \theta) - \sum_{j=1}^{2N_r} \log \mathbb{P}(\mathbf{a}_j | \mathbf{q}_j; \theta). \quad (3)$$

4.4 Fine-tuning and inference of GRIP

To facilitate effective fine-tuning, GRIP divides the fine-tuning into two stages. In the first stage, we only fine-tune the LoRA on graph context memorization. The overall loss for the first stage is:

$$\mathcal{L}_{stage1} = \mathcal{L}_{context} + \mathcal{L}_{summary}. \quad (4)$$

In the second stage, we fine-tune the LoRA on question answering tasks:

$$\mathcal{L}_{stage2} = \mathcal{L}_{QA} \quad (5)$$

Other training details can be found in Appendix A.1.

During inference, the model already encodes the graph context within the LoRA parameters, allowing it to directly solve downstream tasks without requiring the graph as explicit input. Let the downstream task be denoted by $\mathbf{t}$ and the graph context by $\mathbf{g}$. The inference process of GRIP is simply $\mathbf{a} = \text{GRIP}(\mathbf{t})$, whereas standard LLM inference requires $\mathbf{a} = \text{LLM}(\text{concat}(\mathbf{g}, \mathbf{t}))$. Since $\mathbf{g}$ typically forms a long sequence—especially for large-scale graphs—standard LLM inference demands a significantly larger context window and incurs higher computational cost compared to GRIP.

4.5 Additional discussion on GRIP

In this section, we provide additional discussion on GRIP.

The training cost of GRIP. First, although GRIP is more efficient than standard LLMs with graph-to-sequence conversion at inference time, it requires fine-tuning on carefully designed tasks, which introduces additional computational overhead compared

to direct inference. For small-scale graphs, this fine-tuning cost may outweigh the inference savings. However, the practicality of GRIP becomes evident for large-scale graphs, particularly in scenarios involving frequent and repeated queries over the same graph, where the one-time fine-tuning cost can be amortized and leads to substantial overall efficiency gains.

Comparison with RAG-based methods. RAG-based methods are widely used for graph-related tasks, especially for large graphs, but they target a different goal from GRIP. RAG-based methods retrieve question-relevant subgraphs at inference time, whereas GRIP internalizes graph knowledge directly into the LLM parameters. Consequently, RAG-based performance heavily depends on retrieval quality and often requires increasingly complex retrieval and indexing designs [12, 26], while also introducing additional inference latency that typically grows at least linearly with graph size and accumulates under repeated queries. In contrast, GRIP only requires standard LoRA-based fine-tuning and incurs no additional overhead at inference time beyond a single LLM forward pass. Moreover, GRIP is complementary to RAG-based methods and can be naturally integrated to enhance the answering stage by providing implicit global graph knowledge alongside retrieved evidence.

Dealing with multiple graphs or dynamic graphs. GRIP is primarily designed for a single graph, where each graph is stored in an independent LoRA module. Extending GRIP to multiple graphs is nevertheless straightforward, as multiple graphs can be treated as a single graph with multiple connected components and jointly used for fine-tuning. In addition, techniques such as LoRA merging [15] can be leveraged to directly combine knowledge learned from multiple LoRA modules. We leave a systematic study of multi-graph training and LoRA composition to future work. Regarding dynamic graphs, GRIP naturally supports graph growth through standard continual fine-tuning on newly added or updated subgraphs, which will be further discussed in the experimental section. Nevertheless, we acknowledge that GRIP may be less suitable for graphs that undergo frequent and large-scale changes. This limitation, however, is not specific to GRIP, but reflects a more general challenge of maintaining up-to-date knowledge in large foundation models, which is beyond the scope of this work.

5 Experiments

In this section, we conduct various experiments to validate the effectiveness of GRIP. Specifically, we aim to answer the following questions:

Q1: How well does GRIP compare to standard LLMs and existing graph-based models on different graph tasks?

Q2: How efficient is GRIP compared to standard graph-to-sequence methods?

Q3: How effective are the designed fine-tuning tasks in GRIP?

Q4: What are the key hyperparameter settings in GRIP?

Q5: Can GRIP deal with growing graphs?

More details about the implementation and experimental setting can be found in Appendix A and Appendix B, respectively.

Table 2: Performance on the knowledge graphs (IT: input token; OOC: out of context window).

Model	FB15K237 ↑	Avg #IT ↓	WN18RR ↑	Avg #IT ↓
OFA	70.84	1304.90	30.96	1036.48
GOFA	80.69	1304.90	32.89	1036.48
Qwen-7b	79.65	155.74	14.74	156.01
Qwen-7b _{context}	79.19	1330.56	41.19	1624.53
GRIP-qwen-7b	88.75	155.74	64.30	156.01
Llama-8b	56.66	152.75	28.08	155.70
Llama-8b _{context}	44.68	1304.90	53.86	1036.48
GRIP-llama-8b	77.79	152.75	63.26	155.70
Full-context	OOC	>10M	OOC	>4M

5.1 Setup

Dataset. We evaluate GRIP on two categories of graph tasks. The first category is the traditional knowledge graph completion task. We include FB15K237 [2], WN18RR [7], CoDEx-Medium [32], and NELL23K [27]. The task for all these datasets is to predict missing relations between entities. The second category consists of advanced reasoning tasks on graphs. We include Scene Graph [12] and CLEGR-Reasoning [31]. Scene Graph is a dataset in which each graph corresponds to an image, and the task is to answer various questions related to the image. CLEGR-Reasoning contains multiple synthetic subway graphs, and the task is to answer reasoning questions defined over these graphs. For these two multi-graph datasets, each graph is internalised into its own dedicated LoRA. Additional details for each dataset are provided in Appendix B.1.

Model and baselines. To provide a comprehensive evaluation, we adopt both Qwen2.5-7b [43] and Llama3.1-8b [9] as the base LLM for GRIP. For all fine-tuning task generation, we use Qwen2.5-7b [43]. To ensure a fair comparison, we include baselines from the two categories discussed in Section 2: the LLM baselines (Qwen2.5-7b and Llama3.1-8b with edge-list context) represent graph-to-sequence methods, while OFA [25] and GOFA [16] represent graph-to-embedding methods with zero-shot inference. LLM baselines are run by us; OFA/GOFA numbers are taken from the original papers.

Metrics. We use accuracy as the primary evaluation metric for all datasets. To ensure fairness, for the Scene Graph and CLEGR-Reasoning datasets we adopt the LLM-as-Judge approach [20]. Specifically, we use the Qwen2.5-32B model as the judge to determine whether the model outputs are semantically equivalent to the ground-truth labels. For all knowledge graph datasets, accuracy is computed directly using exact match.

5.2 Results on performance comparison

In this section, we present the main experimental results. To provide a more holistic view, for LLM baseline models, we evaluate two variants. In models labeled with the subscript context, the LLM is provided with both the question and the relevant graph context (represented by the edge list). For the knowledge graph datasets, where the entire graph is too large to fit within the LLM’s context window, we sample a subgraph centered around the two target entities for

Table 3: Performance on the knowledge graphs continued (IT: input token; OOC: out of context window).

Model	CoDEX $\uparrow$	Avg #IT $\downarrow$	NELL23K $\uparrow$	Avg #IT $\downarrow$
Qwen-7b	83.09	124.21	77.38	160.80
Qwen-7b _{context}	73.32	1055.29	81.38	590.37
GRIP-qwen-7b	94.42	124.21	87.74	160.80
Llama-8b	71.03	124.61	20.05	160.33
Llama-8b _{context}	47.46	1044.44	53.33	586.15
GRIP-llama-8b	91.58	124.61	77.40	160.33
Full-context	OOC	>5M	OOC	>0.5M

relation prediction, which serves as a retrieval-augmented baseline. Additionally, we include a row labeled full-context to indicate the total number of tokens required to represent the complete knowledge graphs when converted into sequential text. For models without the subscript, only the question is provided as input—identical to the input used for GRIP. Performance on the knowledge graph datasets is reported in Table 2 and Table 3, while the results on the two reasoning datasets are shown in Table 4. Note that for WN18RR, we observe high variance across runs; therefore, we report the average performance over five runs. For the other datasets, results are consistent across runs, so we report the single-run result.

To answer **Q1**, we first observe that GRIP consistently outperforms baseline LLMs without context across all datasets. This demonstrates the effectiveness of GRIP in injecting graph knowledge directly into the LoRA parameters, enabling task solving without explicit graph context. Next, we examine the results on knowledge graph datasets and compare GRIP to models that utilize explicit graph context. We find that GRIP achieves superior performance on nearly all datasets compared to both graph model and LLM model baselines (as indicated by bold scores). This improvement can be attributed to two factors. First, during fine-tuning, **GRIP injects knowledge from the entire graph into the model parameters**—a capability absent in baseline LLM methods, which rely on subgraph sampling to fit within the context window. As a result, GRIP leverages information from the entire graph to better solve downstream tasks. Second, these tasks evaluate the model on link prediction. Since GRIP incorporates similar link prediction tasks through context QA as part of its training objectives, they are effectively closer to in-domain evaluations. We also note that the context-based LLM baseline can occasionally underperform its no-context counterpart (e.g. Qwen-7b on CoDEX): the sampled subgraph around the queried entities is sometimes incomplete or distracting, which can mislead the model.

Next, we turn to the two reasoning tasks in Table 4. These datasets correspond to a small-graph regime in which the full graph fits in the LLM’s context window, so context-based inference is naturally favoured; the goal here is therefore not for GRIP to dominate the full-context baseline, but to remain competitive at much lower inference cost and to further improve over the baseline when used as a test-time adaptation. Although GRIP does not surpass baseline LLMs with full context on these tasks, its performance remains comparable, with only a minimal gap. We would like to emphasize that baseline LLMs achieve these results by inputting the **full graph**

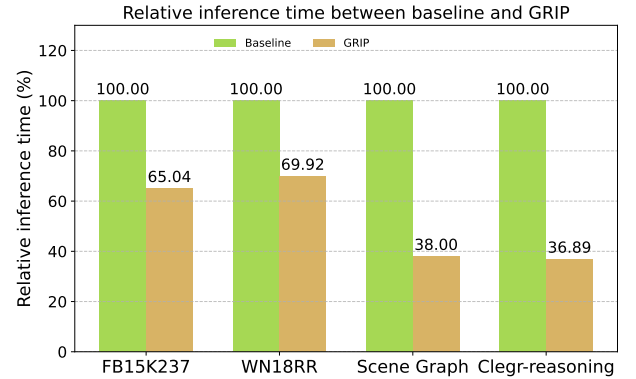
Table 4: Performance on the reasoning datasets (IT: input token; Scene: Scene Graph; Clegr: Clegr-reasoning).

Model	Scene $\uparrow$	Avg #IT $\downarrow$	Clegr $\uparrow$	Avg #IT $\downarrow$
GOFA	34.06	4376.18	-	-
Qwen-7b	3.54	67.94	15.81	106.41
Qwen-7b _{context}	67.46	5295.33	33.48	6338.8
GRIP-qwen-7b	53.27	67.94	27.66	106.41
GRIP-qwen-7b _{context}	68.71	5295.33	31.53	6338.8
Llama-8b	6.96	64.99	0.0	98.58
Llama-8b _{context}	67.06	4376.18	24.76	6213.93
GRIP-llama-8b	53.27	64.99	27.14	98.58
GRIP-llama-8b _{context}	73.99	4376.18	32.75	6213.93

context, while GRIP attains similar accuracy without any context, which is fundamentally different and indicates the strong potential of GRIP. To provide further evidence, we also evaluate GRIP under the same context as baseline LLMs on the reasoning tasks, and we observe that GRIP consistently outperforms the baselines. These results indicate that **GRIP can also serve as an effective test-time training method for enhancing the performance of LLMs on graph tasks**.

In summary, for graphs too large to fit into the context windows of LLMs, GRIP consistently outperforms baseline methods by encoding the entire graph into the model parameters. For graphs that fit within the context window, GRIP achieves comparable performance without any graph context and surpasses baselines when provided with the same context. These results collectively confirm the effectiveness of GRIP.

5.3 Results on the efficiency

**Figure 2: Relative inference time comparison between baseline and GRIP.**

To answer **Q2**, we provide the inference time and its comparison with baseline LLMs and GRIP on multiple datasets using Qwen2.5-7b, as shown in Table 5 and Figure 2. Since GRIP does not require explicit access to the graph context during inference, it achieves substantially lower inference time compared to standard LLMs.

Table 5: Inference time (seconds) on test dataset with Qwen2.5-7b.

dataset	Standard (with context)	GRIP
FB15K237	6316.71	4108.45
WN18RR	1157.69	809.47
Scene Graph	1083.69	411.81
Clegr-reasoning	17122.46	6316.52

Table 6: Cost analysis of GRIP on FB15K237. Training time is one-time only; inference numbers are averaged per query.

Method	Train (s)	Inf. (ms)	Avg. #IT	Speedup
Llama3.1-8B	–	83.41	1326.27	1.00×
GRIP (Llama3.1-8B)	15,850	10.62	155.74	7.85×

Specifically, on knowledge graph datasets where baseline LLMs rely on subgraphs as context, GRIP reduces inference time by about 30%. On reasoning datasets where baseline LLMs require the full graph as context, GRIP reduces inference time by more than 60%. These results further demonstrate the efficiency of GRIP. It is worth noting that GRIP does involve an additional fine-tuning phase. However, once fine-tuned, the resulting LoRA parameters can be stored and reused for any future inference tasks on the same graph. Consequently, the one-time fine-tuning cost becomes negligible compared to the cumulative inference cost, especially in scenarios such as GraphRAG or large-scale web graph applications, where inference is performed repeatedly. To make this concrete, Table 6 reports the wall-clock training and inference cost of GRIP versus the Llama3.1-8B baseline on FB15K237.

5.4 Ablation studies on GRIP design

To answer **Q3**, **Q4** and **Q5**, we conduct multiple ablation studies on GRIP. All experiments in this section are based on the Qwen2.5-7b model.

Effects of fine-tuning task. First, we evaluate the effectiveness of each fine-tuning task in GRIP by varying the numbers of N_s , N_c , and N_r on both the Scene Graph and FB15K237 datasets. The results, shown in Table 7 and Table 8, indicate that removing any task degrades overall performance, directly demonstrating the contribution of each task to GRIP. In particular, for the Scene Graph dataset, removing reasoning QA and summarization tasks results in the largest performance drop, while for FB15K237, the context QA contributes most significantly. This aligns with intuition: Scene Graph requires more complex reasoning, whereas FB15K237 centers on simple link prediction. These findings highlight the versatility of GRIP across different types of graph tasks.

Effects of LoRA module. Next, we examine the LoRA setting in GRIP. In our experiments, we apply LoRA only to the MLP components of the LLM. To validate this choice, we test applying LoRA to the attention modules and to all modules, with results shown in Table 9. The results show that applying LoRA to the MLP modules is most effective. A likely reason is that MLP modules serve as knowledge storage and processing units, while attention

Table 7: Effects of summarization and QA tasks on Scene Graph.

N_s	N_c	N_r	Accuracy $\uparrow$
0	0	0	3.54
0	50	100	45.94
50	0	100	52.54
50	50	0	45.74
50	50	100	53.27

Table 8: Effects of summarization and QA tasks on FB15K237.

N_s	N_c	N_r	Accuracy $\uparrow$
0	0	0	0.1
6000	0	2000	81.34
0	8000	2000	84.09
6000	8000	0	81.62
6000	8000	2000	88.75

modules focus on retrieval. In our setting, knowledge storage is more critical, and the retrieval ability of the base LLM is already sufficient.

Table 9: Ablation study on LoRA adapter position.

Accuracy	ATT	ALL	MLP
Scene Graph	43.10	50.36	53.27

Effects of LoRA rank. We further study the effect of the LoRA rank r , and report the results in Figure 3. Several observations can be made. First, GRIP already achieves strong performance on large knowledge graphs even with a rank of $r = 1$, indicating that the memorized graph knowledge is stored in a highly compressed form. Second, different graphs exhibit different optimal rank settings, and the preferred rank is positively correlated with graph size. In particular, FB15K-237 requires a higher rank (up to $r = 24$) to achieve the best performance, whereas for CoDEx and NELL23K, a small rank of $r = 4$ is sufficient to reach near-optimal performance on downstream tasks. However, the best LoRA rank may depend not only on the size of the graph, but also on the downstream tasks.

Dealing with a growing graph. Finally, we evaluate the performance of GRIP under periodically growing graphs. Specifically, we partition each knowledge graph into three subgraphs using a BFS-based strategy to ensure connectivity. Then, we train a single LoRA module in three consecutive stages with GRIP. At each stage, training tasks are generated only from the newly added subgraph. For a fair comparison, we sample one-third of the training tasks at each stage compared to the full setting, and evaluate the resulting model on the same full evaluation set. The results are reported in Table 10. Overall, GRIP maintains reasonable performance in the multi-stage setting, with accuracy generally improving as more graph parts are incorporated. However, the behavior varies across

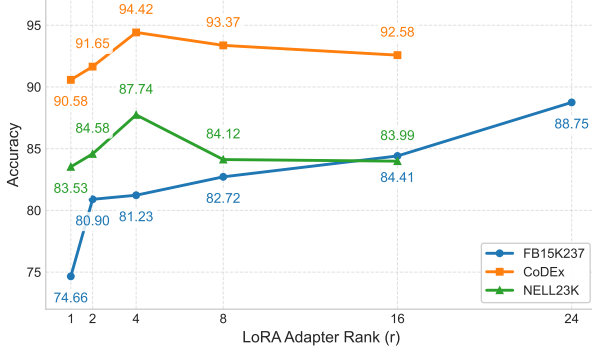


Figure 3: Ablation study on LoRA rank.

Table 10: Performance of GRIP on multi-stage setting. (Standard refers to a single-stage setting.)

Accuracy	stage 1	stage 2	stage 3	standard
NELL23K	68.72	63.57	82.44	87.74
WN18RR	51.91	65.95	69.66	64.30

Table 11: Accuracy of GRIP on Qwen3 backbones.

Backbone	Dataset	No Context	Context	GRIP
Qwen3-8B	WN18RR	25.50	60.79	68.48
	FB15K237	77.03	74.06	91.01
Qwen3-14B	WN18RR	25.37	61.62	84.68
	FB15K237	83.18	87.78	92.99

datasets. On NELL23K, performance is relatively weak in the first two stages and remains below the standard setting even after the third stage, whereas on WN18RR, the multi-stage setting slightly outperforms the standard setting. We attribute this variation mainly to the graph partition strategy, as the distribution of partial graphs may deviate from that of the original graph. In addition, training only on newly added subgraphs at each stage may induce forgetting of previously learned knowledge. We leave the design of more effective multi-stage training strategies for GRIP to future work.

Robustness across LLM backbones. To verify that the benefit of GRIP is not tied to the specific base LLMs used above, we additionally train GRIP on Qwen3-8B and Qwen3-14B; as shown in Table 11, GRIP remains the best variant on both WN18RR and FB15K237 across both model sizes.

6 Conclusion and Limitation

In this work, we propose GRIP, a practical parameterised graph memory framework that adapts pre-trained LLMs to graph-structured data via parameter-efficient fine-tuning and in-parameter knowledge injection. GRIP provides a unified framework for generating fine-tuning tasks that inject graph context into LoRA parameters, enabling the model to reason over graphs without requiring explicit graph context or specialized graph-processing modules at inference time. We evaluate GRIP across multiple datasets and show that,

for large-scale graphs exceeding the LLM’s context window, GRIP consistently outperforms subgraph-based baselines by effectively leveraging the memorized global graph. For graphs that fit within the context window, it achieves comparable performance without explicit context, highlighting efficiency. However, GRIP introduces additional training cost, which may exceed inference cost for small graphs, making it more suitable for large-scale settings. Moreover, GRIP is not well-suited for highly dynamic or frequently updated graphs, which reflects a broader limitation of large foundation models rather than a shortcoming specific to GRIP. We leave these directions to future work.

Acknowledgments

Jiarui Feng, Donghong Cai, and Yixin Chen are supported by a Washington University Transcend Initiative Grant and a DOE SFEE Moonshot grant. Muhan Zhang is supported by the National Natural Science Foundation of China (62550138, 62276003).

References

- [1] Zeyuan Allen-Zhu and Yuanzhi Li. 2023. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316* (2023).
- [2] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating embeddings for modeling multi-relational data. *Advances in neural information processing systems* 26 (2013).
- [3] Andrew Carlson, Justin Betteridge, Bryan Kisiel, Burr Settles, Estevam Hruschka, and Tom Mitchell. 2010. Toward an architecture for never-ending language learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 24. 1306–1313.
- [4] Rujikorn Charakorn, Edoardo Cetin, Yujin Tang, and Robert Tjarko Lange. 2025. Text-to-LoRA: Instant Transformer Adaption. *arXiv preprint arXiv:2506.06105* (2025).
- [5] Runjin Chen, Tong Zhao, Ajay Kumar Jaiswal, Neil Shah, and Zhangyang Wang. 2024. LLaGA: Large Language and Graph Assistant. In *International Conference on Machine Learning*. PMLR, 7809–7823.
- [6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).
- [7] Tim Dettmers, Minervini Pasquale, Stenortorp Pontus, and Sebastian Riedel. 2018. Convolutional 2D Knowledge Graph Embeddings. In *Proceedings of the 32th AAAI Conference on Artificial Intelligence*. 1811–1818. <https://arxiv.org/abs/1707.01476>
- [8] Jiarui Feng, Hao Liu, Lecheng Kong, Mingfang Zhu, Yixin Chen, and Muhan Zhang. 2024. TAGLAS: An atlas of text-attributed graph datasets in the era of large graph and language models. *arXiv preprint arXiv:2406.14683* (2024).
- [9] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [10] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [11] Xiaoxin He, Xavier Bresson, Thomas Laurent, Adam Perold, Yann LeCun, and Bryan Hooi. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- [12] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *Advances in Neural Information Processing Systems* 37 (2024), 132876–132907.
- [13] Yufei He, Yuan Sui, Xiaoxin He, and Bryan Hooi. 2024. UniGraph: Learning a Unified Cross-Domain Foundation Model for Text-Attributed Graphs. *arXiv preprint arXiv:2402.13630* (2024).
- [14] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2021. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- [15] Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. 2023. LoraHub: Efficient cross-task generalization via dynamic lora composition. *arXiv preprint arXiv:2307.13269* (2023).

- [16] Lecheng Kong, Jiarui Feng, Hao Liu, Chongsong Huang, Jiaxin Huang, Yixin Chen, and Muhan Zhang. 2025. GOfA: A Generative One-For-All Model for Joint Graph Language Modeling. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=mJblC9hfm>
- [17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [18] Jakub Lála, Odhran O'Donoghue, Aleksandar Shtedritski, Sam Cox, Samuel G. Rodriques, and Andrew D. White. 2023. Paperqa: Retrieval-augmented generative agent for scientific research. *arXiv preprint arXiv:2312.07559* (2023).
- [19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [20] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. *arXiv:2412.05579* [cs.CL] <https://arxiv.org/abs/2412.05579>
- [21] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*. PMLR, 19730–19742.
- [22] Jintang Li, Ruofan Wu, Yuchang Zhu, Huizhe Zhang, Liang Chen, and Zibin Zheng. 2025. Are Large Language Models In-Context Graph Learners? *arXiv:2502.13562* [cs.LG] <https://arxiv.org/abs/2502.13562>
- [23] Tianqianjin Lin, Pengwei Yan, Kaisong Song, Zhuoren Jiang, Yangyang Kang, Jun Lin, Weikang Yuan, Junjie Cao, Changlong Sun, and Xiaozhong Liu. 2024. LangGFM: A Large Language Model Alone Can be a Powerful Graph Foundation Model. *arXiv preprint arXiv:2410.14961* (2024).
- [24] Aixian Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Cheng-gang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [25] Hao Liu, Jiarui Feng, Lecheng Kong, Ningyue Liang, Dacheng Tao, Yixin Chen, and Muhan Zhang. 2023. One For All: Towards Training One Graph Model For All Classification Tasks. In *The Twelfth International Conference on Learning Representations*.
- [26] Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Dinh Phung, Chen Gong, and Shirui Pan. 2025. GFM-RAG: Graph Foundation Model for Retrieval Augmented Generation. *NeurIPS 2025* (2025).
- [27] Xin Lv, Xu Han, Lei Hou, Juanzi Li, Zhiyuan Liu, Wei Zhang, Yichi Zhang, Hao Kong, and Suhui Wu. 2020. Dynamic anticipation and completion for multi-hop reasoning over sparse knowledge graph. *arXiv preprint arXiv:2010.01899* (2020).
- [28] Yansheng Mao, Jiaqi Li, Fanxu Meng, Jing Xiong, Zilong Zheng, and Muhan Zhang. 2024. LIFT: Improving Long Context Understanding Through Long Input Fine-Tuning. *arXiv preprint arXiv:2412.13626* (2024).
- [29] OpenAI and et al. 2024. GPT-4 Technical Report. *arXiv:2303.08774* [cs.CL] <https://arxiv.org/abs/2303.08774>
- [30] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*. 8024–8035.
- [31] Soham Petkar, Anirudh Vempati, Akshit Sinha, Ponnurangam Kumaraguru, Chirag Agarwal, et al. 2025. A Graph Talks, But Who's Listening? Rethinking Evaluations for Graph-Language Models. *arXiv preprint arXiv:2508.20583* (2025).
- [32] Tara Safavi and Danaï Koutra. 2020. Codex: A comprehensive knowledge graph completion benchmark. *arXiv preprint arXiv:2009.07810* (2020).
- [33] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems* 36 (2023), 38154–38180.
- [34] George Stoica, Pratik Ramesh, Boglarka Ecsedi, Leshem Choshen, and Judy Hoffman. 2024. Model merging with svd to tie the knots. *arXiv preprint arXiv:2410.19735* (2024).
- [35] Weihang Su, Yichen Tang, Qingyao Ai, Junxi Yan, Changyue Wang, Hongning Wang, Ziyi Ye, Yujia Zhou, and Yiqun Liu. 2025. Parametric Retrieval Augmented Generation. *arXiv preprint arXiv:2501.15915* (2025).
- [36] Yuqiao Tan, Shizhu He, Huanxuan Liao, Jun Zhao, and Kang Liu. 2025. Dynamic Parametric Retrieval Augmented Generation for Test-time Knowledge Enhancement. *arXiv:2503.23895* [cs.CL] <https://arxiv.org/abs/2503.23895>
- [37] Jiabin Tang, Yuhao Yang, Wei Wei, Lei Shi, Lixin Su, Suqi Cheng, Dawei Yin, and Chao Huang. 2024. Graphgpt: Graph instruction tuning for large language models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 491–500.
- [38] Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2023. Can language models solve graph problems in natural language? *Advances in Neural Information Processing Systems* 36 (2023), 30840–30861.
- [39] Haoyu Wang, Shikun Liu, Rongzhe Wei, and Pan Li. 2025. Model generalization on text attribute graphs: Principles with large language models. *arXiv preprint arXiv:2502.11836* (2025).
- [40] Yan Wang, Dongyang Ma, and Deng Cai. 2024. With greater text comes greater necessity: Inference-time training helps long text generation. *arXiv preprint arXiv:2401.11504* (2024).
- [41] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. HuggingFace's Transformers: State-of-the-art Natural Language Processing. *arXiv:1910.03771* [cs.CL] <https://arxiv.org/abs/1910.03771>
- [42] Xixi Wu, Yifei Shen, Caihua Shan, Kaitao Song, Siwei Wang, Bohang Zhang, Jiarui Feng, Hong Cheng, Wei Chen, Yun Xiong, et al. 2024. Can Graph Learning Improve Planning in LLM-based Agents?. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [43] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jiahong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yeqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).
- [44] Ruosong Ye, Caiqi Zhang, Runhui Wang, Shuyuan Xu, and Yongfeng Zhang. 2024. Language is All a Graph Needs. In *EACL (Findings)*.
- [45] Mengmei Zhang, Mingwei Sun, Peng Wang, Shen Fan, Yanhu Mo, Xiaoxiao Xu, Hong Liu, Cheng Yang, and Chuan Shi. 2024. GraphTranslator: Aligning Graph Model to Large Language Model for Open-ended Tasks. *arXiv preprint arXiv:2402.07197* (2024).
- [46] Xi Zhu, Haochen Xue, Ziwei Zhao, Wujiang Xu, Jingyuan Huang, Minghao Guo, Qifan Wang, Kaixiong Zhou, and Yongfeng Zhang. 2025. LLM as GNN: Graph Vocabulary Learning for Text-Attributed Graph Foundation Models. *arXiv:2503.03313* [cs.LG] <https://arxiv.org/abs/2503.03313>

A Implementation Details

In this section, we provide our implementation details of GRIP.

A.1 Implementation setting

We implement GRIP based on PyTorch [30], the Hugging Face Transformers library [41], and LIFT [28]. The code for reproducibility is provided in <https://github.com/JiaruiFeng/GRIP>.

For the LoRA adapter, we apply LoRA by default to the MLP modules of each LLM layer. For subgraph sampling in both the summarization and reasoning QA task generation, we first randomly select a root node from the entire graph. We then iteratively expand the subgraph by sampling neighbors at each hop. Specifically, at each hop, we sample up to three neighbors per node and perform this process for three hops. This procedure results in a subgraph containing at most 10 nodes. For efficient task generation, we leverage the vLLM inference framework [17]. For generated fine-tuning tasks, we use plain text for context and summarization tasks and adopt an instruction-tuning template on all QA tasks. Each fine-tuning sample is formatted using the `apply_chat_template` function from the Hugging Face Transformers library. The exact user prompt and answer templates for context memorization, summarization, and edge-level context QA all follow short fixed patterns (e.g. “In context graph {title}, the node {src} is {rel} the node {tgt}.”); the full set is released with the code.

A.2 Prompts Design

We use a single LLM (Qwen2.5-7b) to generate all fine-tuning tasks. Due to space limitation, here we only present the prompt design of

Table 12: Dataset statistics.

Dataset	Avg. # Node	Avg. # Edge	Avg. #Node words	Avg. # Edge words	# G
FB15K237	14,541	310,116	20.1	8.4	1
WN18RR	40,943	93,003	23.3	1.0	1
CoDEx-Medium	17,050	206,205	2.2	1.9	1
NELL23K	22,925	36,358	3.7	4.5	1
Scene Graph	19.13	68.44	20.1	9.8	100000
Clegr-reasoning	26.54	28.32	23.82	21.65	400

Table 13: The hyperparameter settings for all experiments

hyperparameters	Scene Graph	FB15K237	WN18RR	Clegr-reasoning	CoDEx-Medium	NELL23K
lora_r	16	24	24	16	4	4
lora_α	32	48	48	32	8	8
lora components	MLP	MLP	MLP	MLP	MLP	MLP
N_s	50	6000	6000	20	6000	6000
N_r	100	2000	2000	160	2000	2000
N_c	50	8000	8000	20	8000	8000
stage1 minimum epoch	5	1	1	5	1	1
stage1 maximum epoch	50	1	1	50	1	1
stage2 minimum epoch	5	1	1	5	1	1
stage2 maximum epoch	50	5	5	50	10(qwen)/5(llama)	10(qwen)/5(llama)
early stop loss threshold	0.4	0.15	0.15	0.4	0.15	0.15
gradient accumulation steps	full	512	512	full	512	512
learning rate	1e-3	1e-3	1e-3	1e-3	1e-3	1e-3
scheduler	linear	linear	linear	linear	linear	linear
β_1	0.9	0.9	0.9	0.9	0.9	0.9
β_2	0.98	0.98	0.98	0.98	0.98	0.98
ϵ	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4
maximum gradient norm	1.0	1.0	1.0	1.0	1.0	1.0

the summarization task as a representative example. The remaining prompts (rephrasing, node-level / edge-level context QA, and the four reasoning-QA variants: multi-hop, global, binary, and k -shot) follow the same template and are all released with the code at <https://github.com/JiaruiFeng/GRIP>.

Summary: You are given several text snippets, which representing real-world facts. Your task is to generate one summary to rephrase all information while exploring potential relationships across the snippets. Specifically:

- Be accurate and complete, preserving ALL factual details like color or properties and numeric details like coordinate.
- rewrite the original context in a completely different way, tone, writing style, and logic to summarize the original context.
- Using rewriting techniques, including reordering, exchanging active and passive sentences, or synonym replacement.
- If one text snippet, only do the summarization and rephrase.
- If multiple text snippets are provided, focus on the relations between different text sentences, like multi-entity connections.

The text snippets are provided below:
{context}
Please answer in the following format: Summary: [summary].
Please DON'T output quotes.

B Experiment details

B.1 Dataset details

In this section, we describe the details of all datasets we used for evaluation. The statistics of all datasets can be found in Table 12. The raw data of Scene Graph, FB15k237, and WN18RR datasets are obtained from TAGLAS [8]. The Clegr-reasoning [31], CoDEx-Medium [32], and NELL23K [27] are obtained from its original source.

Scene Graph. Scene Graphs is a graph question answering dataset on scene graphs. Each graph in SceneGraphs contains objects connected by the relationship between two objects. It contains 59,978/19,997/20,025 graph samples for the train/val/test sets. Due to the time limitation, we only evaluate all methods on the first 500 test samples.

Clegr-reasoning. Clegr-reasoning is a graph question answering generated by a syntactic subway line graph. Each graph contains several subway lines with intersections. It contains 400 different graphs and each graph has 34 questions generated from 34 different templates. These question templates test the model on different parts of graph reasoning, including topology, path, aggregation, and filtering. An example question template is: Which station is adjacent to both {Station} and {Station}.

FB15K237 FB15K237 is a knowledge graph. The dataset contains 14,541 nodes and 310,116 relations. Nodes in the dataset are entities in the knowledge graph and edges represent the relation between two entities. It contains 237 different relation types. There are a total of 272,115/17,535/20,466 samples for train/val/test sets, respectively. For simplicity, in evaluation, we only evaluate the randomly sampled 10000 samples from the test set, and each sample is formalized as a 10-way classification task. That is, the question will ask the model to select the correct one relation from 10 candidate relations, instead of all 237 relations.

WN18RR WN18RR is another knowledge graph extracted from WordNet. It contains 40,943 nodes and 93,003 relations, where each node is an English word and each edge represents the relation between two words. It contains 11 different relation types and 86,835/3,034/3,134 samples for train/val/test sets, respectively. For WN18RR, we evaluate on the whole test set.

CoDEx-Medium CoDEx-Medium is a knowledge graph built from Wikipedia/Wikidata. It contains 17,050 nodes and 206,205 relations, where each node is a Wikipedia/Wikidata-aligned entity and each edge denotes a factual relation between two entities. The dataset includes 51 relation types, with 185,584/10,310/10,311 relations in the train/val/test sets, respectively. Following the FB15K237

setup, we formalize each test set sample as a 10-way classification task, while we evaluate on the entire test set.

NELL23K NELL23K is another knowledge graph constructed from the Never-Ending Language Learning (NELL) project [3]. It contains 22,925 nodes and 36,358 relations. Each node corresponds to a NELL concept instance (e.g., concept:person, concept:city), and each edge denotes a typed relation between two concepts. The dataset includes 200 different relation types, with 25,445/4,961/4,952 relations in the train/val/test sets, respectively. Following the CoDEx-Medium protocol, we cast each test sample as a 10-way classification task and evaluate on the entire test set.

B.2 Training process and hyperparameters

All experiments can be conducted on a single NVIDIA A100-SXM4-80GB GPU. To accelerate the training and inference process, we use 4 GPUs in total and split all datasets into 4 subsets and process them independently on each GPU. In Table 13, we provide hyperparameter settings for all datasets.

For all knowledge graph datasets, the graph used in every GRIP stage (context memorisation, summarisation, and QA generation) contains only the training edges; validation and test edges are removed before any subgraph sampling or QA construction.